FPT UNIVERSITY - HO CHI MINH CITY CAMPUS

EMBEDDED LINUX COURSE



FPT UNIVERSITY

ASSIGNMENT 01 REPORT

EMBEDDED LINUX

Topic: “HOW TO WRITE A DEVICE DRIVER”

CHARACTER DEVICE AND MISC COUNTER

Student: Nguyễn Huy Hoàng

Student ID: SE203347

Target: QEMU vexpress-a9 / ARMv7 / Linux 5.15.0

THÀNH PHỐ HỒ CHÍ MINH - THÁNG 10 NĂM 2026

LỜI MỞ ĐẦU

Bài tập “How to Write a Device Driver” tạo cơ hội đi từ lời gọi read, write và ioctl của chương trình C đến cơ chế dispatch trong VFS, rồi quan sát trạng thái bên trong Linux kernel. Việc chạy module trên QEMU ARMv7 giúp kiểm tra cùng lúc mã kernel, ABI dùng chung và root filesystem tối giản.

Báo cáo trình bày thiết kế và bằng chứng kiểm thử cho hai module riêng: character driver hỗ trợ ioctl và misc counter device. Phần thực nghiệm dùng transcript do sinh viên ghi trực tiếp trên máy khi hai script kiểm thử chạy trong QEMU. Các lỗi phát sinh ở Makefile và khởi động rootfs được ghi rõ cùng biện pháp sửa.

TÓM TẮT

Bài gồm hai kernel module độc lập. Task 1 đăng ký character device tại major 241, trao đổi byte và bốn lệnh ioctl. Task 2 đăng ký misc device với major 10, đếm số nguyên qua read/write và công bố thống kê trong /proc/counter_info. Các chương trình test C được biên dịch tĩnh cho ARM, chạy trong QEMU cùng kernel 5.15.0 đã build module.

PHẠM VI BẰNG CHỨNG

Các giá trị phần thực nghiệm được lấy từ qemu_session.txt. Phiên test được chạy trên rootfs trước khi thêm /etc/init.d/rcS; sinh viên mount /proc, /sys và /dev trong shell. Một lần boot sau đã xác nhận dòng “Assignment 01 rootfs ready” từ initramfs sửa đổi, qua ảnh màn hình riêng. Vì log phiên test không chứa boot sau, báo cáo không gộp hai lần chạy thành một.

MỤC LỤC

Trang	Nội dung
4	1. Mục tiêu và kiến trúc tổng thể
5	2. Task 1 - thiết kế character driver
6	3. Task 1 - ioctl, an toàn bộ nhớ và kiểm thử
7	4. Task 2 - misc counter và procfs
8	5. Task 2 - đồng bộ, offset và kiểm thử
9	6. Task 3 - so sánh và quyết định thiết kế
10	7. Task 4 - bốn câu hỏi lý thuyết
11	8. Kết quả thực nghiệm và khắc phục lỗi
12	9. Kết luận, tái lập và tài liệu tham khảo
13-14	Phụ lục A - log Task 1, dmesg
15	Phụ lục B - log Task 2, procfs, dmesg

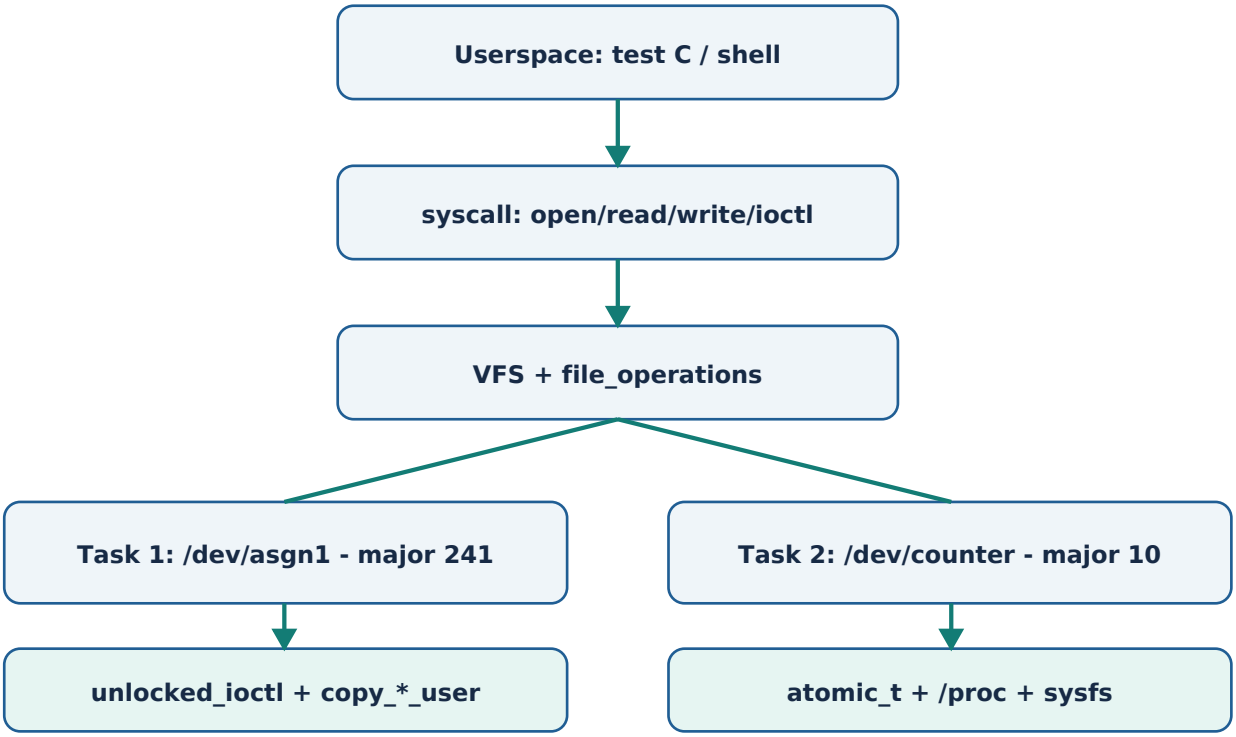
DANH MỤC HÌNH VÀ BẢNG

Hình 1. Kiến trúc chung của hai module trong QEMU - trang 4.

Bảng 1. Thành phần các lớp và điểm quan sát - trang 4. Các bảng tiếp theo tóm tắt ioctl, trạng thái counter, kết quả kiểm thử và so sánh driver trong từng chương.

1. Mục tiêu và kiến trúc tổng thể

Mục tiêu là đi hết tuyến dữ liệu từ chương trình userspace qua syscall, VFS, bảng file_operations tới trạng thái trong kernel, rồi quay trở lại userspace. Device node nối tên file với cặp major:minor. Major chọn driver; minor phân biệt thiết bị trong cùng driver.



Hình 1. Kiến trúc chung của hai module trong QEMU.

Lớp	Task 1	Task 2
Điểm vào user	/dev/asgn1; asgn1_test, shell	/dev/counter; counter_test, shell
VFS	open, read, write, unlocked_ioctl	open, read, write
Đăng ký	register_chrdev; major 241	misc_register; major 10, minor động
Quan sát	dmesg; ioctl GET_STATS	/proc/counter_info; dmesg

Nhánh ioctl trong hình đi từ userspace qua ioctl() syscall, VFS và file_operations.unlocked_ioctl đến handler Task 1; sau xử lý, copy_from_user hoặc copy_to_user chuyển argument qua ranh giới kernel/user. Với Task 2, read/write tới atomic counter, còn /proc/counter_info là giao diện quan sát đi từ VFS/procfs. Task 1 đăng ký device class trong sysfs. Mutex bảo vệ trạng thái chung qua nhiều lệnh gọi.

2. Task 1 - character driver

Trạng thái và đăng ký

Module asgn1_driver đăng ký major cố định 241, tạo lớp thiết bị và device để /dev/asgn1 có thể xuất hiện khi hệ thống có cơ chế quản lý node. Trên rootfs tối giản, script còn có bước mknod dự phòng. Khi gỡ module, driver giải phóng các thành phần theo thứ tự ngược. Trạng thái chung gồm buffer 1024 byte, độ dài hiện tại, mode và năm trường thống kê.

Trường	Ý nghĩa và quy ước
open_count	Tổng lượt mở device; RESET giữ lại
write_count	Số write hợp lệ; RESET đưa về 0
read_count	Số read trả dữ liệu; EOF không tính
buffer_len	Số byte hiện có, bao gồm tiền tố nếu mode 0
last_write_size	Số byte nhận từ userspace, không bao gồm tiền tố

Đường đi của read/write

write lấy dữ liệu user vào buffer kernel và thay nội dung cũ. Mode 1 lưu nguyên dữ liệu; mode 0 thêm chuỗi [KERNEL] trước dữ liệu, có kiểm tra sức chứa. read dựa trên *ppos: copy tối đa phần còn lại, tăng offset sau khi copy thành công, rồi trả 0 khi đến cuối. Do đó lệnh cat dừng đúng ở EOF.

Bất biến cần giữ

• 0 ≤ buffer_len ≤ 1024; thống kê và buffer được cập nhật dưới mutex.

• Không tăng read_count khi read trả 0; không tăng write_count nếu copy từ user thất bại.

• Mode mặc định bằng 1; SET_MODE chỉ chấp nhận 0 hoặc 1.

3. Task 1 - ioctl và kiểm thử

Lệnh	Định nghĩa trong header chung	Tác động
RESET_BUFFER	_IO('A',0)	Xóa buffer và thống kê đọc/ghi
GET_STATS	_IOR('A',1,struct asgn1_stats)	Trả 5 số nguyên
SET_MODE	_IOW('A',2,int)	0: thêm prefix; 1: echo
GET_VERSION	_IOR('A',3,char[64])	Trả phiên bản trong vùng 64 byte

Các macro mã hóa magic, số lệnh, chiều và kích thước, nhưng không tự chuyển dữ liệu. .unlocked_ioctl chọn request, sao chép qua ranh giới user/kernel và trả -ENOTTY cho request lạ. SET_MODE sai trả -EINVAL. Header dùng chung giữ định nghĩa ABI của kernel và chương trình C khớp nhau.

Ví dụ ASGN1_GET_STATS dùng magic A, số lệnh 1: _IOC_NR(...) = 1, _IOC_SIZE(...) = sizeof(struct asgn1_stats) = 20 byte trên ARMv7 (5 số int). GET_VERSION dùng vùng 64 byte nên _IOC_SIZE(...) = 64. Magic phân nhóm request để hạn chế va chạm tên lệnh; driver vẫn kiểm tra toàn bộ command mã hóa.

Trích mã nguồn thực tế: copy dữ liệu ra userspace

```
return copy_to_user(user, &stats, sizeof(stats)) ? -EFAULT : 0;
```

Log QEMU: các nhánh chính

```
GET_VERSION: asgn1_driver v1.0 - SE203347
SET_MODE: 0 (prefix)
READ OK: [[KERNEL]abc], length=11
GET_STATS: open_count=5 write_count=2 read_count=2
           buffer_len=11 last_write_size=3
SET_MODE: 1 (echo)
INVALID MODE: EINVAL OK
READ OK: [xyz], length=3

RESET_BUFFER: called
AFTER RESET: open_count=8 write_count=0 read_count=0
             buffer_len=0 last_write_size=0
UNKNOWN COMMAND: ENOTTY OK
TASK 1 TEST PASSED
TASK 1 SCRIPT PASSED
```

buffer_len=11 gồm 8 byte [KERNEL] cộng 3 byte abc; RESET giữ open_count nhưng xóa thống kê đọc/ghi.

4. Task 2 - misc counter và procfs

counter_driver dùng misc_register với major chung 10; kernel chọn minor đang trống. Phiên chạy QEMU ghi nhận minor 125, đây là giá trị động và không nên hard-code. Device /dev/counter xuất số thập phân kèm newline. proc_create tạo /proc/counter_info; seq_file và single_open trình bày counter, total_reads, total_writes.

Một lần mở, một snapshot

read đầu tiên trong một lần mở lấy giá trị counter, tăng counter đúng một lần và giữ chuỗi snapshot trong trạng thái riêng của file. Những lần read tiếp theo của cùng file lấy phần còn lại theo *ppos, rồi EOF. Cơ chế này vẫn đúng khi userspace đọc từng byte; nếu tăng ở mỗi lời gọi read, một lệnh cat có thể làm bộ đếm nhảy nhiều lần.

Write có kiểm tra

write copy chuỗi ASCII từ user vào buffer cục bộ, phân tích với kstrtoint, từ chối giá trị âm hoặc không phải số. Chỉ write hợp lệ mới đặt lại counter và tăng total_writes. Lệnh ghi 0 là phép reset giá trị counter; thống kê total_reads và total_writes vẫn được giữ.

Thao tác	Giá trị quan sát từ log
Trước write	counter=0; total_reads=0; total_writes=0
Sau hai lần đọc	Trả lần lượt 0, 1; counter tiến đến 2
Ghi 100	counter=100; total_reads=2; total_writes=1
Ghi 0	counter=0; total_reads=3; total_writes=2
Dữ liệu -1 và abc	Bị từ chối; total_writes vẫn bằng 2

5. Task 2 - đồng bộ và kiểm thử offset

Vì sao có cả atomic_t và mutex?

Nếu counter là int thường, hai process có thể cùng đọc 7, cùng tính 8 rồi cùng ghi 8: mất một lần tăng. atomic_t giúp atomic_inc không mất cập nhật ở phép tăng đơn lẻ. Tuy nhiên "đọc giá trị cũ, chuẩn bị snapshot, tăng counter, cập nhật thống kê" gồm nhiều bước; mutex bao toàn bộ chuỗi này để hai process không cùng quan sát một giá trị cũ hoặc xen một write vào giữa. Mutex cũng bảo vệ các thống kê đi kèm.

Bằng chứng đọc từng phần

```
counter_driver: write set=100
counter_driver: read value=100 next=101
PARTIAL READ OK: line=[100\n], counter=101 reads=5 writes=3
TASK 2 SCRIPT PASSED
```

counter_test đọc chuỗi "100\n" thành các mảnh nhưng total_reads tăng đúng một lượt và counter sau đó bằng 101. Trên procfs, sau lần reset và một read tiếp theo, log ghi counter=1, total_reads=4, total_writes=2. Hai input âm và chữ không làm thay đổi số write hợp lệ.

Chuỗi sự kiện của lần đọc

Thời điểm	Trạng thái
open	Tạo trạng thái riêng cho file, chưa tăng counter
read lần đầu	Chụp số 100, tăng thành 101, total_reads +1
read phần còn lại	Trả byte còn lại từ snapshot; không tăng tiếp
read khi hết	Trả 0 (EOF)
release	Giải phóng trạng thái riêng

6. TASK 3 - So sánh ba lớp driver Linux

Tiêu chí	Character	Block	Network
Đơn vị	Dòng byte/lệnh	Sector/block	Frame/packet
Giao diện	file_operations	block_device_operations; request queue	net_device_ops
Điểm vào	/dev và syscall file	/dev, filesystem	Socket/network stack
Ví dụ	UART, asgn1, counter	eMMC, SD card	Ethernet, Wi-Fi
Bộ đệm	Tùy driver và VFS	Page cache, block queue	sk_buff, RX/TX queue

Áp dụng vào bài

asgn1 và counter không trình bày không gian sector cũng không trao đổi packet. Chúng cung cấp một stream byte, lệnh điều khiển và trạng thái phần mềm, nên thuộc nhóm character. misc device vẫn là một character device: nó rút gọn phần đăng ký và dùng major 10 chung, chứ không tạo ra lớp I/O thứ tư.

Task 1 dùng register_chrdev với major 241 cố định: có quyền chọn major và quản lý class/device node, đổi lại phải xử lý va chạm major và dọn từng tài nguyên khi lỗi hoặc unload. Task 2 dùng misc_register: kernel cấp minor động dưới major 10 và gom bước đăng ký vào một miscdevice, phù hợp thiết bị counter nhỏ. Không được lấy minor 125 trong một lần chạy làm hằng số cho các lần chạy sau.

Hai tình huống chọn driver

• UART truyền byte theo thứ tự: chọn character interface; có thể dùng tầng TTY nếu cần đầy đủ hành vi terminal.

• eMMC lưu root filesystem bằng địa chỉ sector: chọn block layer. Raw NAND có erase block, bad block và OOB nên thường dùng MTD, không đối xử trực tiếp như eMMC.

Giới hạn của ví dụ

Hai thiết bị của assignment là thiết bị mô phỏng trong kernel; không nói chuyện với thanh ghi phần cứng, IRQ hay DMA. Bài tập nhấn mạnh ABI userspace, vòng đời module, đồng bộ và phép thử trên target ARM.

7. TASK 4 - Bốn câu hỏi lý thuyết

Câu 1. Reference count tăng/giảm và buộc unload

Khi file_operations.owner = THIS_MODULE, kernel giữ một tham chiếu trong thời gian file của driver đang mở và trả khi đóng; module_get/module_put cho phép các đường code tự tăng/giảm tham chiếu bổ sung, chỉ khi lấy tham chiếu thành công. rmmod bình thường sẽ thất bại nếu module đang được giữ. rmmod -f chỉ có thể ép gỡ với quyền phù hợp và kernel hỗ trợ CONFIG_MODULE_FORCE_UNLOAD; có thể để lại lời gọi tới code đã tháo, gây lỗi nghiêm trọng. Cách an toàn là tìm process đang mở device, đóng file và rmmod bình thường. Trong bài này script kiểm thử làm theo cách đó.

Câu 2. copy_to_user và MMU

Con trỏ ioctl là địa chỉ ảo của process; page table của process xác định ánh xạ trang và quyền ghi. Một địa chỉ có thể chưa được map, không có quyền, hoặc bị thay đổi; kernel không thể giả định memcpy trực tiếp từ kernel tới con trỏ ấy là thao tác hợp lệ. Truy cập sai có thể sinh page fault trong kernel hoặc vi phạm bảo vệ bộ nhớ. copy_to_user thực hiện truy cập user có xử lý fault và trả số byte chưa copy; driver trả -EFAULT nếu không sao chép đủ struct.

Câu 3. MTD và block subsystem

Block layer thấy thiết bị theo sector và hỗ trợ hàng đợi request, page cache, filesystem block. MTD biểu diễn flash raw: page, erase block, bad block và đôi khi OOB; ghi đè byte tùy ý không tương đương ghi sector. Raw NAND nối trực tiếp với controller là ví dụ cần MTD (thường kết hợp UBI/UBIFS); eMMC che giấu flash qua controller nội bộ và xuất sector, nên đi block layer.

Câu 4. VFS read, *ppos và điều kiện dừng cat

cat gọi read(fd, buf, n) nhiều lần. Mỗi lần VFS gọi file_operations.read với offset của file đang mở; giá trị read > 0 nghĩa là còn dữ liệu để in, bằng 0 là EOF và cat dừng, âm là lỗi. Nếu driver luôn trả lại chuỗi số và không kiểm tra/tăng *ppos, cat sẽ tiếp tục gọi read và in vô hạn. counter_driver chụp “100\n” vào trạng thái của lần open, cập nhật offset khi copy, và trả 0 khi offset chạm độ dài; auto-increment xảy ra đúng một lần cho snapshot.

8. Thực nghiệm: build, boot, lỗi và sửa

Mục	Bằng chứng
Target	Linux 5.15.0, ARMv7, QEMU vexpress-a9
Compiler trong boot log	arm-linux-gnueabihf-gcc 11.4.0
Module	asgn1 major 241; counter major 10 minor 125
Kết quả	TASK 1 SCRIPT PASSED; TASK 2 SCRIPT PASSED
Log gốc	logs/qemu_session.txt từ chương trình script trên Ubuntu

Lỗi build và rootfs thực tế

Lần build ban đầu gặp lỗi lệnh ARM atomic/isb/ldrex do Makefile module thiếu cờ kiến trúc. Bổ sung ccflags-y += -march=armv7-a ở cả hai Makefile rồi build lại; sinh viên đã xác nhận bốn artifact ELF ARM 32-bit, hai module có vermagic 5.15.0 SMP mod_unload ARMv7 p2v8 qua ảnh terminal.

Rootfs sao chép thực tế có inittab gọi /dev/tty2, /dev/tty3, /dev/tty4; QEMU -nographic báo không mở được các TTY này. Sau khi chuyển inittab sang askfirst trên console, shell lên nhưng /etc/init.d/rcS thiếu trong image đó. Trong phiên test, sinh viên mount /proc, /sys, /dev thủ công; cả hai script PASS. Sau đó thêm rcS vào bản rootfs assignment, đóng lại initramfs và boot lần nữa; ảnh boot xác nhận “Assignment 01 rootfs ready”.

Đánh giá giới hạn bằng chứng

Transcript cung cấp output kiểm thử và dmesg của hai driver. Lần boot sau khi sửa rcS được chứng minh bằng ảnh terminal, không có trong transcript test đã nộp. Không suy diễn thêm benchmark hiệu năng, thử tải song song hoặc đo phần cứng vì chưa có dữ liệu đó.

9. Kết luận và tái lập

Hai module đã biên dịch cho ARMv7 và được thử trên QEMU/Linux 5.15.0. Task 1 thể hiện ioctl ABI, hai mode, thống kê, reset và đường lỗi; Task 2 thể hiện misc registration, counter nguyên tử, snapshot theo open, procfs và xử lý input không hợp lệ. Hai script kết thúc PASSED. Phiên boot sau chứng minh rcS đã được khôi phục trong initramfs.

Cấu trúc nộp khuyến nghị

```
SE203347_NguyenHuyHoang_Asgn01/
  task1_ioctl/   asgn1_driver.c, asgn1_ioctl.h, asgn1_test.c, Makefile, *.ko
  task2_misc/     counter_driver.c, counter_test.c, Makefile, *.ko
  output/         initramfs_assignment.cpio.gz
  logs/           test_asgn1_output.txt, test_counter_output.txt, dmesg_full.txt
  bao_cao/        SE203347_NguyenHuyHoang_Asgn01_BaoCao.pdf
  README.md
```

Để tái lập: build hai thư mục với cùng cây kernel 5.15; chép hai .ko, hai chương trình ARM và hai shell script vào rootfs; đóng gói initramfs bằng cpio newc; boot qemu-system-arm -M vexpress-a9 với zImage và DTB cùng kernel; chạy hai script trong guest. Đối chiếu uname -r với modinfo vermagic trước insmod.

Tài liệu tham khảo

[1] Đề ASSIGNMENT01_EmbeddedLinux.docx.pdf và nguồn LAB01/LAB02 do sinh viên cung cấp.

[2] Linux Kernel Documentation, “ioctl based interfaces”, https://docs.kernel.org/driver-api/ioctl.html.

[3] Linux Kernel Documentation, “Miscellaneous Devices”, https://docs.kernel.org/5.10/driver-api/misc_devices.html.

[4] Linux Kernel Documentation, “Atomic types”, https://docs.kernel.org/core-api/wrappers/atomic_t.html.

[5] Mã nguồn hai module, hai test C và log QEMU của assignment này.

PHỤ LỤC A - Log Task 1 trong QEMU

Trích nguyên chuỗi output của test_asgn1.sh từ insmod đến rmmod; log gốc đầy đủ được lưu ở logs/qemu_session.txt. Dòng ioctl và dmesg do script xuất trong cùng phiên.

```
=== TASK 1: ioctl character driver ===
asgn1_driver: loading out-of-tree module taints kernel.
asgn1: loaded major=241 mode=1
crw-------    1 0        0         241,   0 Oct 10 19:07 /dev/asgn1
[1] Default echo mode: write and read
asgn1: ioctl unknown command=0x5401
asgn1: write 5 bytes, mode=1, stored=5
asgn1: read 5 bytes
read: [hello]
[2] ioctl tests, both modes and reset
asgn1: ioctl GET_VERSION
GET_VERSION: asgn1_driver v1.0 - SE203347
asgn1: ioctl SET_MODE
SET_MODE: 0 (prefix)
asgn1: write 3 bytes, mode=0, stored=11
asgn1: read 11 bytes
READ OK: [[KERNEL]abc], length=11
asgn1: ioctl GET_STATS
GET_STATS: open_count=5 write_count=2 read_count=2 buffer_len=11 last_write_size=3
asgn1: ioctl SET_MODE
SET_MODE: 1 (echo)
asgn1: ioctl SET_MODE
INVALID MODE: EINVAL OK
asgn1: write 3 bytes, mode=1, stored=3
asgn1: read 3 bytes
READ OK: [xyz], length=3
asgn1: ioctl RESET_BUFFER
RESET_BUFFER: called
READ OK: [], length=0
asgn1: ioctl GET_STATS
AFTER RESET: open_count=8 write_count=0 read_count=0 buffer_len=0 last_write_size=0
asgn1: ioctl unknown command=0x4163
UNKNOWN COMMAND: ENOTTY OK
TASK 1 TEST PASSED
[3] Buffer must be empty after RESET
empty buffer OK
```

PHỤ LỤC A - Tiếp: gỡ module và dmesg Task 1

```
[4] Unload and inspect cleanup log
asgn1: unloaded
asgn1: loaded major=241 mode=1
asgn1: ioctl unknown command=0x5401
asgn1: write 5 bytes, mode=1, stored=5
asgn1: read 5 bytes
asgn1: ioctl GET_VERSION
asgn1: ioctl SET_MODE
asgn1: write 3 bytes, mode=0, stored=11
asgn1: read 11 bytes
asgn1: ioctl GET_STATS
asgn1: ioctl SET_MODE
asgn1: ioctl SET_MODE
asgn1: write 3 bytes, mode=1, stored=3
asgn1: read 3 bytes
asgn1: ioctl RESET_BUFFER
asgn1: ioctl GET_STATS
asgn1: ioctl unknown command=0x4163
asgn1: unloaded
TASK 1 SCRIPT PASSED
```

PHỤ LỤC B - Log Task 2 trong QEMU

Ba bản đọc /proc/counter_info thể hiện trước write, sau write 100 và sau write 0. Các dòng dmesg sau khi gỡ module nằm ở cuối output dưới đây.

```
=== TASK 2: misc counter device ===
counter_driver: loaded major=10 minor=125
crw-------    1 0        0          10, 125 Oct 10 19:07 /dev/counter
[1] Before write
counter: 0
total_reads: 0
total_writes: 0
counter_driver: read value=0 next=1
counter_driver: read value=1 next=2
read sequence: 0, 1
[2] Set 100
counter_driver: write set=100
counter: 100
total_reads: 2
total_writes: 1
counter_driver: read value=100 next=101
read after write: 100
[3] Reset to zero
counter_driver: write set=0
counter: 0
total_reads: 3
total_writes: 2
counter_driver: read value=0 next=1
read after reset: 0
[4] Reject negative and invalid input
invalid inputs rejected
counter: 1
total_reads: 4
total_writes: 2
[5] Partial reads: one increment per open
counter_driver: write set=100
counter_driver: read value=100 next=101
PARTIAL READ OK: line=[100\n], counter=101 reads=5 writes=3
counter_driver: unloaded
counter_driver: loaded major=10 minor=125
counter_driver: read value=0 next=1
counter_driver: read value=1 next=2
counter_driver: write set=100
counter_driver: read value=100 next=101
counter_driver: write set=0
counter_driver: read value=0 next=1
counter_driver: write set=100
counter_driver: read value=100 next=101
counter_driver: unloaded
TASK 2 SCRIPT PASSED
```

File logs/dmesg_excerpt.txt tách các dòng driver từ transcript để tiện tra cứu; đó là trích đoạn, không phải toàn bộ kernel ring buffer.